



Université de Lyon

Université Jean Monnet Saint-Étienne · Mines Saint-Étienne - Institut Mines Télécom

Master in Computer Science - Cyber-Physical and Social Systems: AI and IoT (CPS2)

Representation of Physical Quantities on the Semantic Web

M2 Internship Thesis submitted in partial fulfillment
of the requirements for the degree of
Master 2 in Computer Science

Irfan Ullah

Academic year 2024-2026

Mandatory research internship conducted at

Département Informatique et Systèmes Intelligents (ISI)
Institut Henri Fayol - Mines Saint-Étienne

1 February 2026 - 30 June 2026

Supervised by

Antoine Zimmermann

Professor
Head of ISI Department
Institut Henri Fayol
École des Mines de Saint-Étienne
LIMOS - UMR CNRS 6158
antoine.zimmermann@emse.fr

Maxime Lefrançois

Associate Professor
Institut Henri Fayol
École des Mines de Saint-Étienne
LIMOS - UMR CNRS 6158
maxime.lefrancois@emse.fr

June 28, 2026

Abstract

Physical quantities are fundamental to scientific, engineering, and medical data on the Semantic Web. The dominant approach is ontology-based: Quantities, Units, Dimensions and Data Types Ontologies (QUDT) and Ontology of units of Measure (OM) provide quantity kinds, unit hierarchies, dimensional vectors, and conversion factors. But four problems persist regardless of ontological richness. The most serious is computational inaccessibility: dimensional structure is encoded in the graph but never reaches the SPARQL Protocol and RDF Query Language (SPARQL) expression evaluator, so comparisons, ordering, and aggregation over mixed-unit data produce physically incomplete results without warning. The second is bounded coverage: derived units multiply combinatorial and no enumeration can be complete. The third is interoperability: independently developed ontologies assign different Internationalized Resource Identifiers (IRIs) to the same physical unit, requiring manual cross-ontology mappings. The fourth is verbosity: a single quantity value requires at least three triples.

The Linked Datatypes (LINDT) specification addresses all four problems at the datatype level through `cdt:ucum`, which encodes a complete quantity as a single typed literal using Unified Code for Units of Measure (UCUM). Because UCUM is a grammar rather than a list, coverage is unlimited by construction. The value space recognises dimensional equivalence natively, and operator semantics expose unit-aware equality, comparison, and arithmetic directly within SPARQL. The only existing implementation is a fork of Apache Jena and it is outdated, and no implementation exists for any other major Resource Description Framework (RDF) library. No general, reproducible procedure for building one has been documented.

This thesis addresses that gap. It presents a five-step methodology covering candidate library identification, extension point analysis, feasibility assessment, implementation, and validation. The feasibility step is an explicit go/no-go decision point before any code is written, and its output records not only whether each obligation can be met but through what mechanism and at what stability cost. A hard constraint frames the assessment: the implementation must leave the host library unmodified.

The methodology is validated through four working implementations across Apache Jena in Java, `rdflib.js` in TypeScript, `dotNetRDF` in C#, and `RDFLib` in Python, each paired with a UCUM backend suited to its ecosystem and accompanied by a test suite written against a single shared reference checklist. The central finding is that how far an extension can reach into a host from outside determines what the implementation can deliver. That reach is set by two factors: the host library's architecture and the host language.

The results confirm that the Custom Datatypes (CDT) approach is technically viable across heterogeneous programming ecosystems and provides a lightweight, standards-compatible path to unit-aware query evaluation on the Semantic Web.

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Antoine Zimmermann and Maxime Lefrançois, for their guidance throughout this internship. Their expertise in semantic web technologies and in the design of the CDT specification shaped this work in every chapter. They were always available to discuss ideas, challenge assumptions, and point me in the right direction.

I thank the Département Informatique et Systèmes Intelligents (ISI) at Mines Saint-Étienne for welcoming me as an intern and providing a stimulating research environment.

I am also grateful to the Cyber Physical and Social Systems (CPS2) programme, jointly operated by Université Jean Monnet Saint-Étienne and Mines Saint-Étienne - Institut Mines Télécom, for the academic foundation that made this work possible.

This work would not have existed without the open-source community. The four implementations described in this thesis are built entirely on openly developed libraries and tools, including Apache Jena, rdflib.js, RDFLib, dotNetRDF, and the supporting libraries that make UCUM unit processing possible. The maintainers and contributors of these projects gave their time freely, and this research depended on that generosity at every step.

Finally, I am deeply grateful to my family for their unconditional support and encouragement throughout my studies. Their patience and belief in me made this work possible.

List of Abbreviations

CDT Custom Datatypes

CPS2 Cyber Physical and Social Systems

IRI Internationalized Resource Identifier

LINDT Linked Datatypes

NASA National Aeronautics and Space Administration

OGC Open Geospatial Consortium

OM Ontology of units of Measure

QUDT Quantities, Units, Dimensions and Data Types Ontologies

RDF Resource Description Framework

SOSA Sensor, Observation, Sample, and Actuator

SPARQL SPARQL Protocol and RDF Query Language

SSN Semantic Sensor Network

UCUM Unified Code for Units of Measure

W3C World Wide Web Consortium

Contents

Abstract	i
Acknowledgements	ii
List of Abbreviations	ii
1 Introduction	1
1.1 Physical Quantities	1
1.2 The Problems	1
1.3 cdt:ucum as a Datatype-Level Solution	2
1.4 Prior Work and the Gap This Thesis Addresses:	2

1. Introduction

1.1 Physical Quantities

Applications in many industry sectors rely on quantity values with units of measure, for sensor observations, actuation, design calculations and simulations, quantitative general knowledge, and clinical data. A temperature reading, for instance, is merely a bare number until it is paired with a specific unit such as degrees Celsius or Kelvin. The representation of such values is therefore critical: a numeric literal alone provides insufficient context, and associating each value correctly with its unit is a precondition for interoperable computation. To achieve this, the value and its unit must be encoded together, and any system that operates on them must respect the dimensional structure of the unit space. A precise foundation is therefore needed to define what quantities are, how they are classified, and how they combine.

The International Vocabulary of Metrology (VIM, JCGM 200:2012) makes the notion precise [1]. It defines a quantity as a property of a phenomenon, body, or substance whose magnitude can be expressed as a number and a reference, where a reference can be a measurement unit. The definition separates two ideas that everyday usage tends to blur. A quantity kind, such as length or temperature, is general and qualitative. A quantity value is specific: the height of one particular object, stated as a number multiplied by a unit. Dimensional analysis governs how quantity values combine. Two quantities can be compared or aggregated only if they share the same dimension. Two measurements of the same dimension expressed in different units denote the same quantity kind. A system that treats them distinct is dimensionally wrong. A system that operates across them without conversion is arithmetically wrong. Representing physical quantities correctly therefore means carrying magnitude and unit together and respecting the dimensional structure of the unit at every step.

In software, this discipline is enforced by unit-aware libraries. Pint in Python and Indriya in Java are representative examples, and comparable libraries exist in most programming languages. In these libraries a quantity is a magnitude paired with a unit. An operation between incommensurable quantities raises an error at runtime, while compatible quantities are converted automatically when combined. Their scope is computation inside a single application. What these libraries do not address is how quantities are stored, serialized, and queried across distributed systems. Hall and Kuster identify representational difficulties with quantities and units in digital systems in general and propose a central metrological register to address them [2].

On the Semantic Web the established answer is ontology-based. QUDT is an ontology, which grew out of a National Aeronautics and Space Administration (NASA) initiative, organises units within a hierarchy of quantity kinds and dimensions and records conversion factors. OM is also an ontology which provides a comparable unit vocabulary with finer-grained quantity kind hierarchies. Sensor, Observation, Sample, and Actuator (SOSA) and Semantic Sensor Network (SSN), published as a joint World Wide Web Consortium (W3C) and Open Geospatial Consortium (OGC) standard, model the observation itself and delegate the encoding of value and unit to QUDT or OM. Keil and Schindler [3] evaluated eight major unit ontologies and found coverage gaps and correctness issues across all of them.

1.2 The Problems

Consider a dataset that records distances from several sources. A construction survey reports in inches, a road network in miles, and a scientific instrument in metres. A SPARQL query asking which measurements exceed one kilometre ought to be straightforward. It is not. The query engine sees bare numbers next to unit IRIs and has no mechanism for treating them as commensurable quantities. Four problems lie behind this failure. Section 3 examines each in detail; this section states them in outline.

The first and most serious is computational inaccessibility. Ontologies such as QUDT and OM do

encode dimensional structure and conversion factors, but that information never reaches the SPARQL expression evaluator. Comparison, ordering, and aggregation operate on raw magnitudes, so queries across mixed units return physically incorrect results without warning. Lefrançois and Zimmermann [4] confirm that, they are not aware of any existing SPARQL engine provides unit-aware evaluation for QUDT or OM quantities. The only workaround is to make conversion factors explicit inside every query. At the time of that work, no such engine existed, and to the best of our knowledge that remains the case today.

The second is bounded coverage. An ontology enumerates its units one by one in a curated graph, yet the space of valid unit expressions has no natural boundary. Derived units multiply combinatorially: every pairing of a length unit with a time unit is a distinct speed unit, and the same holds for pressure, energy, and power. No enumeration can be complete.

The third is interoperability. Independently developed ontologies assign different IRIs to the same physical unit, and their vocabularies overlap only partially [3]. Combining datasets from different sources typically requires cross-ontology mappings, which must be built manually and may not exist for a given pair of ontologies.

The fourth is verbosity. Encoding one quantity value costs several triples. In a dataset with thousands of observations the overhead grows linearly with the data while adding no expressiveness.

1.3 cdt:ucum as a Datatype-Level Solution

The LINDT Custom Datatypes specification approaches these four problems from a different level: not the ontology, but the RDF datatype. It defines `cdt:ucum`, a datatype that encodes a complete quantity value as a single typed literal such as `"1.5 km"^^cdt:ucum`. The unit is part of the literal's lexical form, written in the UCUM grammar. A single triple suffices; no blank node is required. The specification defines operator semantics for equality, comparison, and arithmetic, exposed through native SPARQL syntax, and its value space recognises dimensional equivalence, so two literals denoting the same physical quantity are value-equal regardless of the unit used to express them. Because UCUM is a grammar rather than a list, any unit the grammar can generate is valid. Coverage is unlimited by construction and independent of any curated vocabulary. None of this is achievable with ontology-based representations alone. Section 3 presents the specification and its formal datatype model in full.

1.4 Prior Work and the Gap This Thesis Addresses:

Lefrançois and Zimmermann [4] presented `cdt:ucum` at ESWC 2018 together with an implementation called `jena-ucum`. That implementation is a fork of the Apache Jena repository built against version 3, rather than a standalone extension. Using it means running the forked distribution instead of adding a dependency to a standard Jena installation. No other implementation exists for any other major RDF library, and no general procedure for producing one has been documented.

Research Question: This thesis addresses the following research question: How can the implementation of `cdt:ucum` in an RDF library be generalized into a procedure that is applicable and reproducible across different RDF libraries?

Contributions: This thesis makes three contributions. The first is a five-step methodology for implementing `cdt:ucum` in any RDF library. The second is four working implementations spanning Java (Apache Jena), TypeScript (`rdflib.js`), Python (`RDFLib`), and C# (`dotNetRDF`), each accompanied by a test suite that reflects what that library and UCUM backend combination can and cannot achieve. The third is an empirical analysis of how RDF library architecture and UCUM backend capabilities shape implementation strategy, documenting the limitations encountered in each case study and the patterns that recur across them.

Outline of Remaining Sections: Section 2 covers background on RDF, typed literals, and SPARQL. Section 3 surveys the representation of physical quantities on the Semantic Web, covering ontology-based approaches, the UCUM code system, and the CDT specification. Section 4 reviews prior custom datatype implementations and the candidate RDF and UCUM libraries. Section 5 presents

the five-step methodology. Section 6 reports the four implementations and their findings. Section 7 concludes with a summary of contributions, stated limitations, and directions for future work.

References

- [1] BIPM JCGM et al. *International vocabulary of metrology - basic and general concepts and associated terms (VIM)*. 2012.
- [2] BD Hall and M Kuster. “Representing quantities and units in digital systems”. In: *Measurement: Sensors* 23 (2022), p. 100387.
- [3] Jan Martin Keil and Sirko Schindler. “Comparison and evaluation of ontologies for units of measurement”. In: *Semantic Web* 10.1 (2018), pp. 33–51.
- [4] Maxime Lefrançois and Antoine Zimmermann. “The unified code for units of measure in RDF: cdt: ucum and other UCUM datatypes”. In: *European Semantic Web Conference*. Springer. 2018, pp. 196–201.